

Activity 7: Persistence and Error Recovery

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K4, A4

Estimated time: 75 minutes

Objective

Persist domain snapshots safely and prove recovery from invalid data and write failure.

Deliverable

Versioned JSON repository, migration fixture and recovery tests

Tools

C++ filesystem, JSON fixture, CTest

Included files

- `starter/snapshot-v1.json`
- `starter/repository-contract.hpp`
- `solution/migration.cpp`
- `solution/recovery-test.txt`

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/07-persistence-error-recovery` and validate the supplied fixture with `python3 -m json.tool starter/snapshot-v1.json`. Expect `schemaVersion` 1 and amount values expressed as integer cents.
2. Inspect `starter/repository-contract.hpp`. Confirm the domain depends on the abstract `ExpenseRepository` port rather than a filesystem path or Foundation API.
3. Create `workspace/`, copy the repository contract and fixture, and add `JsonExpenseRepository.cpp` behind the port used by the domain service.
4. Implement save as a two-file sequence: serialise the complete snapshot to `.tmp`, verify the write succeeded, then replace the current snapshot. Never truncate the current file before a complete replacement exists.

5. Add ``schemaVersion`` to every saved document. Keep amounts as integer cents and identities as stable strings.
6. Implement the v1 migration shown in ``solution/migration.cpp``: load ``limit``, translate each expense, apply current defaults and return a current-schema snapshot.
7. Write a restart test that saves two accepted expenses, constructs a new repository instance and asserts the same IDs and cent values are restored.
8. Write a corruption test using invalid JSON. Expect a stable ``corruptStore`` result rather than a crash or an empty-success snapshot.
9. Write a failed-save test with a deterministic repository double. Confirm the previously valid snapshot remains readable and the UI-facing draft is not discarded.
10. Validate every JSON fixture with ``python3 -m json.tool <file>`` and keep test paths under a temporary Activity directory, never the real home or application-support directory.
11. Record the restart, migration, corruption and failed-write results in ``workspace/recovery-evidence.txt``. Compare the expected recovery sequence with ``solution/recovery-test.txt``.
12. Delete only temporary test stores and ``.tmp`` files after evidence capture. Retain both schema fixtures and every migration/recovery test.

Verification

Restart restores accepted data; corrupt input and failed writes become stable recoverable errors.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If tests touch the real home directory, inject a temporary path and fail the test on escape.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Delete only temporary test stores; retain migration fixtures.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.